

MODULE 03 OF 09



Scripting Fundamentals

YOU'LL SHIP

Working lava and vanishing platforms in Coin Rush.

3.1 Script vs LocalScript vs ModuleScript

3.2 Variables and types

3.3 Conditions and loops

3.4 Functions

3.5 Events: build a lava floor

3.6 Tables

Now the world starts reacting. In this module you'll learn Luau, Roblox's scripting language, and use it to make your first interactive hazards: a lava floor that actually hurts, and platforms that disappear.

3.1 Script vs LocalScript vs ModuleScript

Roblox has three kinds of script, and picking the right one is half the battle.

Type	Runs on	Typical place	Use it for
Script	The server	ServerScriptService, or inside a part	Game rules, damage, saving data, rewards
LocalScript	One player's device	StarterPlayerScripts, StarterGui, ReplicatedFirst	UI, camera, input, effects only that player sees
ModuleScript	Whoever <code>require</code> s it	ReplicatedStorage, ServerScriptService	Shared code and settings

A simple rule: **anything that matters, like health, coins, winning or buying, runs on the server.** The client only shows things and sends requests. You'll see why in Module 5.

To create one: right-click **ServerScriptService** → **Insert Object** → **Script**. Rename it right away.

```
print("Coin Rush server started")
```

Press Play and open **View** → **Output**. Your message appears there. The Output window is where every error shows up, so keep it open always.

3.2 Variables and types

A variable is a named box for a value. In Luau you create one with `local`:

```
local playerName = "Alex"      -- string (text)
local coins = 0                -- number
local isAlive = true           -- boolean (true/false)
local nothing = nil            -- nil means "no value"

coins = coins + 10             -- change it later
print(playerName, "has", coins, "coins")
```

Luau also lets you write types, which helps Studio catch mistakes before you run the game:

```
local speed: number = 16
local title: string = "Coin Rush"
```

Instances are values too. You get them with `game`, `workspace` and `script`:

```
local lava = workspace:WaitForChild("LavaFloor")
lava.Color = Color3.fromRGB(255, 90, 20)
lava.Transparency = 0.1
```

TIP Use `WaitForChild` for things that might not have loaded yet. It waits until the object exists instead of erroring.

3.3 Conditions and loops

Conditions choose what happens:

```
local coins = 120

if coins >= 100 then
    print("You can buy the speed upgrade!")
elseif coins >= 50 then
    print("Almost there.")
else
    print("Keep collecting.")
end
```

Comparison operators: `==` equal, `~=` not equal, `<`, `>`, `<=`, `>=`. Combine with `and`, `or`, `not`.

Loops repeat things:

```
-- Count down from 3
for i = 3, 1, -1 do
    print(i)
    task.wait(1)
end
print("GO!")

-- Repeat while a condition is true
local timeLeft = 10
while timeLeft > 0 do
    timeLeft -= 1
    task.wait(1)
end
```

`task.wait(seconds)` pauses the script. Always put a wait inside a `while true do` loop, or Studio will freeze.



Functions

A function is a reusable block of code with a name:

```
local function formatCoins(amount: number): string
    return amount .. " coins"
end

print(formatCoins(25)) --> 25 coins
```

Functions keep your code short and readable. If you write the same three lines twice, turn them into a function.

```
local function setPlatformVisible(platform: BasePart, visible: boolean)
    platform.Transparency = if visible then 0 else 0.8
    platform.CanCollide = visible
end
```



Events: build a lava floor

Events are how Roblox tells your script that *something happened*: a part was touched, a player joined, a button was clicked. You `Connect` a function to an event, and Roblox calls it every time.

Put this **Script** inside `LavaFloor` :

```

local lava = script.Parent

lava.Touched:Connect(function(hit: BasePart)
    local character = hit.Parent
    local humanoid = character and character:FindFirstChildOfClass("Humanoid")
    if humanoid then
        humanoid.Health = 0
    end
end)

```

What's happening:

1. `Touched` fires whenever any part touches the lava.
2. `hit` is the part that touched it, like a player's foot.
3. The foot's parent is the character model. If it has a **Humanoid**, it's a player (or NPC).
4. Setting `Health = 0` knocks them out, and they respawn.

Vanishing platforms. Put this Script in a platform. It disappears shortly after being touched, then comes back:

```

local platform = script.Parent
local busy = false -- a "debounce": ignore touches while we're already running

platform.Touched:Connect(function(hit)
    if busy then return end
    if not hit.Parent:FindFirstChildOfClass("Humanoid") then return end
    busy = true

    task.wait(0.4)
    platform.Transparency = 0.8
    platform.CanCollide = false

    task.wait(2)
    platform.Transparency = 0
    platform.CanCollide = true
    busy = false
end)

```

WATCH OUT `Touched` fires many times per second while a player stands on a part. Without the `busy` debounce, the code would start dozens of times at once.

3.6 Tables

Tables hold many values. They're Luau's lists *and* dictionaries.

```
-- A list (array)
local colors = { "Red", "Blue", "Green" }
print(colors[1])      --> Red (lists start at 1)
print(#colors)        --> 3

for index, color in colors do
    print(index, color)
end

-- A dictionary
local upgradePrices = {
    Speed = 100,
    Jump = 150,
    Magnet = 300,
}
print(upgradePrices.Speed)    --> 100
upgradePrices.Shield = 500    -- add a new key

for name, price in upgradePrices do
    print(name, "costs", price)
end
```

Settings module. Put all Coin Rush tuning values in one ModuleScript in ReplicatedStorage called `GameConfig`:

```
local GameConfig = {
    RoundLength = 90,          -- seconds
    IntermissionLength = 15,
    CoinValue = 1,
    LavaDamage = 100,
    UpgradePrices = {
        Speed = 100,
        Jump = 150,
    },
}

return GameConfig
```

Any script can now read it:

```
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local GameConfig = require(ReplicatedStorage.WaitForChild("GameConfig"))

print("Rounds last", GameConfig.RoundLength, "seconds")
```

Changing the balance of your whole game now means editing one file.

TRY IT

1. Make three vanishing platforms with different timings (read the timings from `GameConfig`).
2. Change the lava so it only removes 40 health per touch, with a one-second debounce per player. (Hint: store the last hit time per character in a table.)

Module recap

- ☐ You know which script type runs where
- ☐ You can use variables, `if`, `for`, `while` and functions
- ☐ The lava floor knocks players out
- ☐ Vanishing platforms work with a debounce
- ☐ `GameConfig` ModuleScript holds your tuning values